CIA-2 PROJECT SUBMISSION

ADVANCED JAVASCRIPT FRONTEND FRAMEWORKS (ANGULAR)

TOPIC: PRODUCT CATALOG AND SHOPPING CART SYSTEM

CLASS: 4BTCSDS

INSTITUTION NAME: CHRIST (DEEMED TO BE UNIVERSITY), BANGALORE

SUBMITTED BY:
AGNES ANNA ROY (2462310)
IRENE AJAY K (2462329)
SHREYA ELIZABETH JOSEPH (2462364)
VIKASINI CHANDRAKUMAR (2462373)

DATE OF SUBMISSION:23/01/2026

“Product Catalog and Shopping Cart System”

❖ Abstract:

This project involves the development of a dynamic Product Catalog and Shopping Cart application using Angular and TypeScript. It simulates the core features of an e-commerce platform, allowing users to browse products, view detailed information, and manage items within a shopping cart. The application is built using Angular’s component-based and reactive architecture, emphasizing modular design, efficient data handling, and maintainable code. Through this project, essential front-end development concepts such as routing, services, data binding, and Angular Material integration are effectively demonstrated.

❖ Project Objective:

The objective of this project is to develop a dynamic front-end application using Angular and TypeScript while applying Angular’s component-based architecture and core TypeScript features for structured and reliable coding. The project also aims to implement key e-commerce functionalities such as product browsing, cart management, and order handling, along with using data binding, directives, services, and dependency injection, and integrating Angular Material to create a responsive and user-friendly interface.

❖ Tools and Technologies:

- Framework: Angular (v17 or later)
- Programming Language: TypeScript
- UI Library: Angular Material
- Development Tools:
    - Node.js
    - Angular CLI
    - Visual Studio Code
- Mock Backend:
    - JSON Server or static JSON files for product and cart data

❖ Key Features:

- Dynamic Product Catalog: Displays a list of available products with real-time rendering
- Product Detail View: Shows detailed information about individual products
- Shopping Cart Management: Allows users to add, remove, and update items in the cart
- Component-Based Design: Ensures modular, reusable, and maintainable code
- Reactive Architecture: Uses Angular services and observables for efficient data handling
- User-Friendly Interface: Enhanced UI using Angular Material components
- Routing Support: Enables smooth navigation between different application views

❖ Approach:

Task1: Setup and TypeScript Fundamentals:
The Product Catalog and Shopping Cart application was developed using Angular as the front-end framework and TypeScript as the core programming language. The project was initialized using Angular CLI, which provided a standardized project structure and simplified development and build processes.

To implement TypeScript fundamentals, key entities of the application were defined using TypeScript interfaces and classes, such as:

- Product
- Category
- CartItem

Strong typing was enforced through the use of appropriate data types, interfaces, access modifiers, and object-oriented principles. This approach ensured type safety, improved code organization, enhanced reusability, and maintained a clean and consistent codebase throughout the application.

Task 2: Angular Architecture and Component Design:
The application follows Angular's component-based architecture, where the user interface and functionality are divided into independent, reusable components. Each component manages a specific feature of the application along with its corresponding logic.

The project structure includes Angular components, HTML templates, CSS styling files, and TypeScript logic files arranged in a modular folder structure. Major components developed include the Product List Component for displaying products, Product Detail Component for showing detailed information, Shopping Cart Component for managing selected items, and Navigation Component for seamless routing between views.

This architectural design improves application scalability, readability, maintainability, and allows efficient reuse of components across different parts of the application.

❖ Screenshots:

Figure 1: Project root folder structure

```
shopping-cart-system/
│
├── src/
│   │
│   ├── app/
│   │   │
│   │   ├── components/
│   │   │   ├── navbar/              # Navigation bar component
│   │   │   ├── home/                # Home / landing page
│   │   │   ├── product-list/        # Displays list of products
│   │   │   ├── product-detail/      # Individual product details view
│   │   │   ├── shopping-cart/       # Shopping cart management
│   │   │   └── checkout/            # Order / checkout page
│   │   │
│   │   ├── models/
│   │   │   ├── product.model.ts     # Product interface & class
│   │   │   ├── category.model.ts    # Category interface & class
│   │   │   └── cart-item.model.ts   # Cart item interface & class
│   │   │
│   │   ├── services/
│   │   │   ├── product.service.ts   # Product data handling
│   │   │   ├── cart.service.ts      # Cart management logic
│   │   │   └── order.service.ts     # Order processing logic
```

```
│   │   │
│   │   ├── app.routes.ts          # Application routing configuration
│   │   ├── app.component.ts       # Root component
│   │   ├── app.component.html     # Root template
│   │   ├── app.component.css      # Root styles
│   │
│   ├── assets/
│   │   ├── images/                # Product images
│   │   └── data/
│   │       └── products.json      # Mock product data
│   │
│   ├── styles.css                 # Global styles
│   ├── main.ts                    # Application entry point
│
├── angular.json                   # Angular CLI configuration
├── tsconfig.json                  # TypeScript configuration
├── package.json                   # Project dependencies
├── package-lock.json              # Dependency lock file
└── README.md                      # Project documentation
```

Figure 2: Source folder structure

Shopping_system_ShopHub / src /

hackershay Start repository

Name
..
components
context
lib
App.tsx
index.css
main.tsx
vite-env.d.ts

Figure 3: Application folder structure

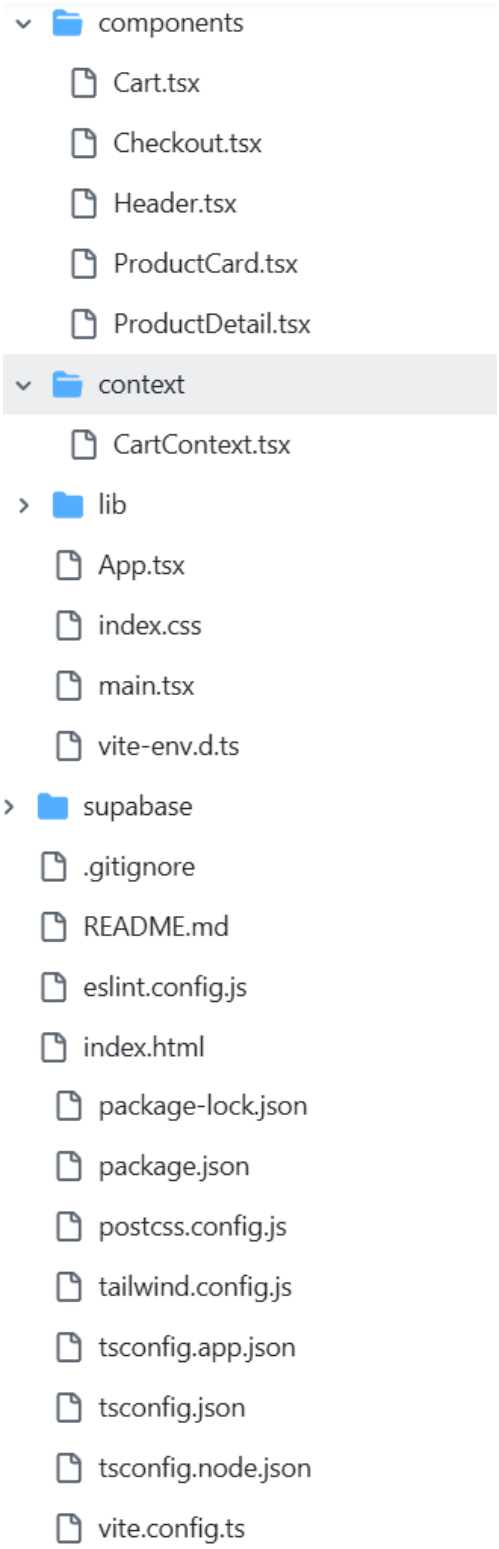


Figure 4: App.ts

Shopping_system_ShopHub / src / App.tsx

hackershay Start repository

bf038b5 · 5 hours ago History

Code Blame 138 lines (123 loc) · 4.23 KB

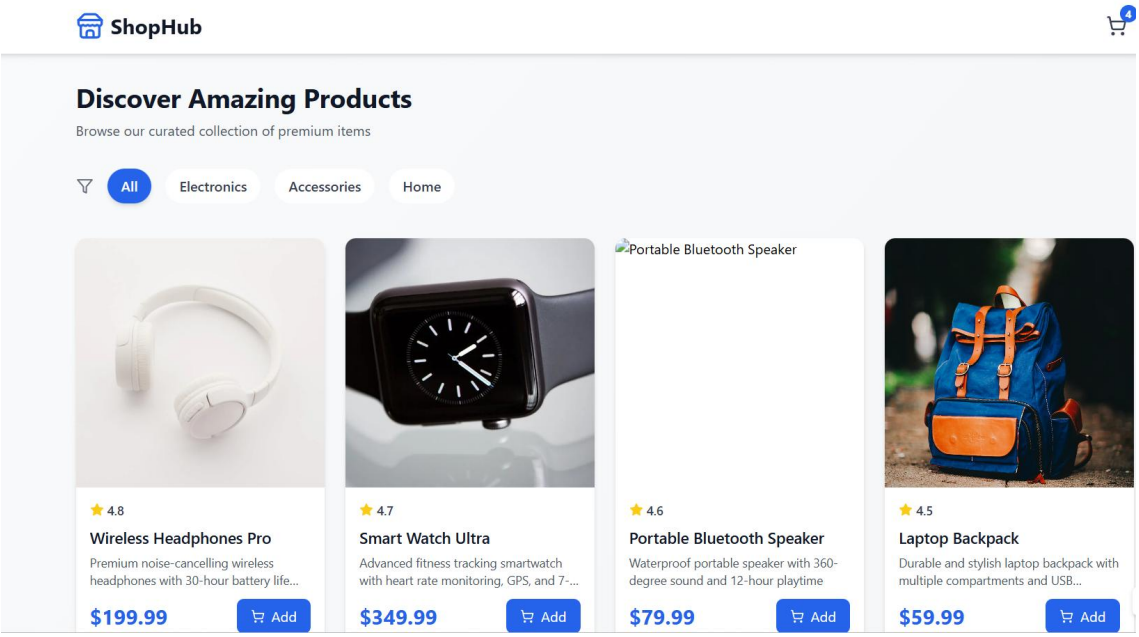
Raw

```
1   import { useState, useEffect } from 'react';
2   import { CartProvider } from './context/CartContext';
3   import { Header } from './components/Header';
4   import { ProductCard } from './components/ProductCard';
5   import { ProductDetail } from './components/ProductDetail';
6   import { Cart } from './components/Cart';
7   import { Checkout } from './components/Checkout';
8   import { supabase } from './lib/supabase';
9   import type { Product } from './lib/database.types';
10  import { Filter } from 'lucide-react';
11
12  function AppContent() {
13    const [products, setProducts] = useState<Product[]>([]);
14    const [filteredProducts, setFilteredProducts] = useState<Product[]>([]);
15    const [selectedProduct, setSelectedProduct] = useState<Product | null>(null);
16    const [isCartOpen, setIsCartOpen] = useState(false);
17    const [isCheckoutOpen, setIsCheckoutOpen] = useState(false);
18    const [selectedCategory, setSelectedCategory] = useState<string>('all');
19    const [isLoading, setIsLoading] = useState(true);
20
21    useEffect(() => {
22      fetchProducts();
23    }, []);
24
25    useEffect(() => {
26      if (selectedCategory === 'all') {
27        setFilteredProducts(products);
28      } else {
29        setFilteredProducts(
30          products.filter((p) => p.category === selectedCategory)
31        );
32      }
33    }, [selectedCategory, products]);
34
35  const fetchProducts = async () => {
36    try {
37      const { data, error } = await supabase
38        .from('products')
39        .select('*')
40        .order('created_at', { ascending: false });
41
42      if (error) throw error;
43      setProducts(data || []);
44      setFilteredProducts(data || []);
45    } catch (error) {
46      console.error('Error fetching products:', error);
47    } finally {
48      setIsLoading(false);
49    }
50  };
51
52  const categories = [
53    'all',
54    ...Array.from(new Set(products.map((p) => p.category))),
55  ];
56
57  return (
```

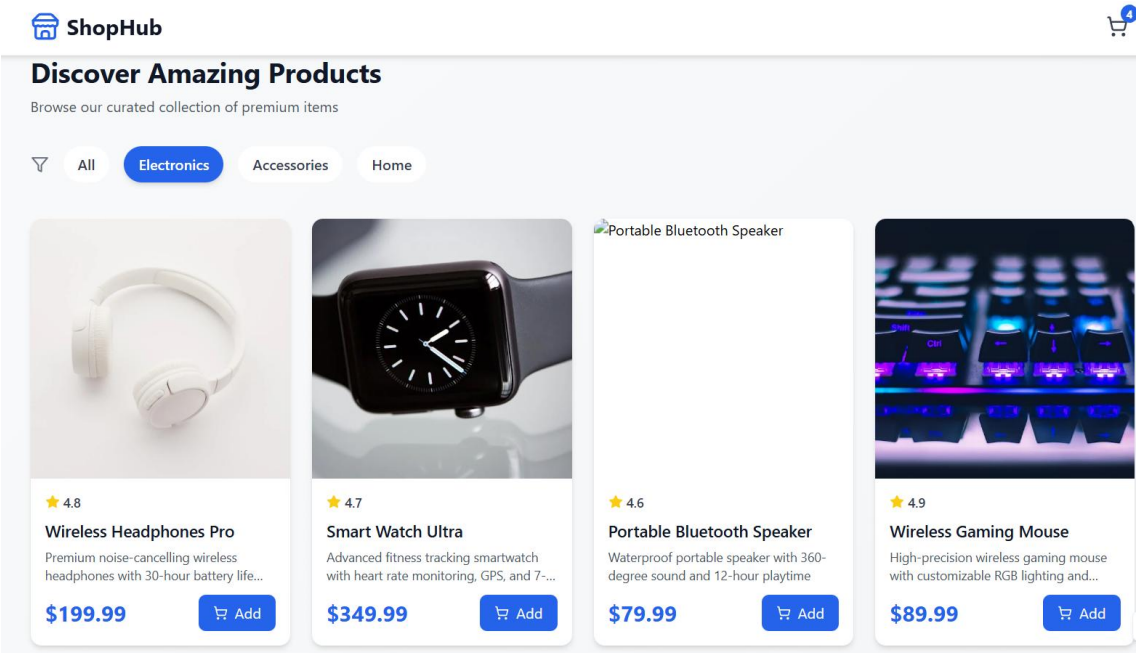
```
12    function AppContent() {
35      const fetchProducts = async () => {
58        <div className="min-h-screen bg-gradient-to-br from-gray-50 to-gray-100">
59          <Header onCartClick={() => setIsCartOpen(true)} />
60
61          <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
62            <div className="mb-8">
63              <h2 className="text-3xl font-bold text-gray-900 mb-2">
64                Discover Amazing Products
65              </h2>
66              <p className="text-gray-600">
67                Browse our curated collection of premium items
68              </p>
69            </div>
70
71            <div className="mb-8 flex items-center gap-4 overflow-x-auto pb-2">
72              <Filter className="w-5 h-5 text-gray-500 flex-shrink-0" />
73              {categories.map((category) => (
74                <button
75                  key={category}
76                  onClick={() => setSelectedCategory(category)}
77                  className={`px-4 py-2 rounded-full font-medium whitespace-nowrap transition-all ${
78                    selectedCategory === category
79                      ? 'bg-blue-600 text-white shadow-md'
80                      : 'bg-white text-gray-700 hover:bg-gray-100'
81                  }`}
82                >
83                  {category.charAt(0).toUpperCase() + category.slice(1)}
84                </button>
85              ))}
86            </div>
88            {isLoading ? (
89              <div className="flex justify-center items-center h-64">
90                <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div>
91              </div>
92            ) : filteredProducts.length === 0 ? (
93              <div className="text-center py-16">
94                <p className="text-gray-500 text-lg">No products available yet.</p>
95              </div>
96            ) : (
97              <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-6">
98                {filteredProducts.map((product) => (
99                  <ProductCard
100                    key={product.id}
101                    product={product}
102                    onViewDetails={setSelectedProduct}
103                  />
104                ))}
105              </div>
106            )}
107          </main>
108
109          {selectedProduct && (
110            <ProductDetail
111              product={selectedProduct}
112              onClose={() => setSelectedProduct(null)}
113            />
114          )}
115
116          <Cart
117            isOpen={isCartOpen}
118            onClose={() => setIsCartOpen(false)}
119            onCheckout={() => setIsCheckoutOpen(true)}
120          />
121
122          <Checkout
123            isOpen={isCheckoutOpen}
124            onClose={() => setIsCheckoutOpen(false)}
125          />
126        </div>
127      );
128    }
129
130    function App() {
131      return (
132        <CartProvider>
133          <AppContent />
134        </CartProvider>
135      );
136    }
137
138    export default App;
```

Figure 5: Application output

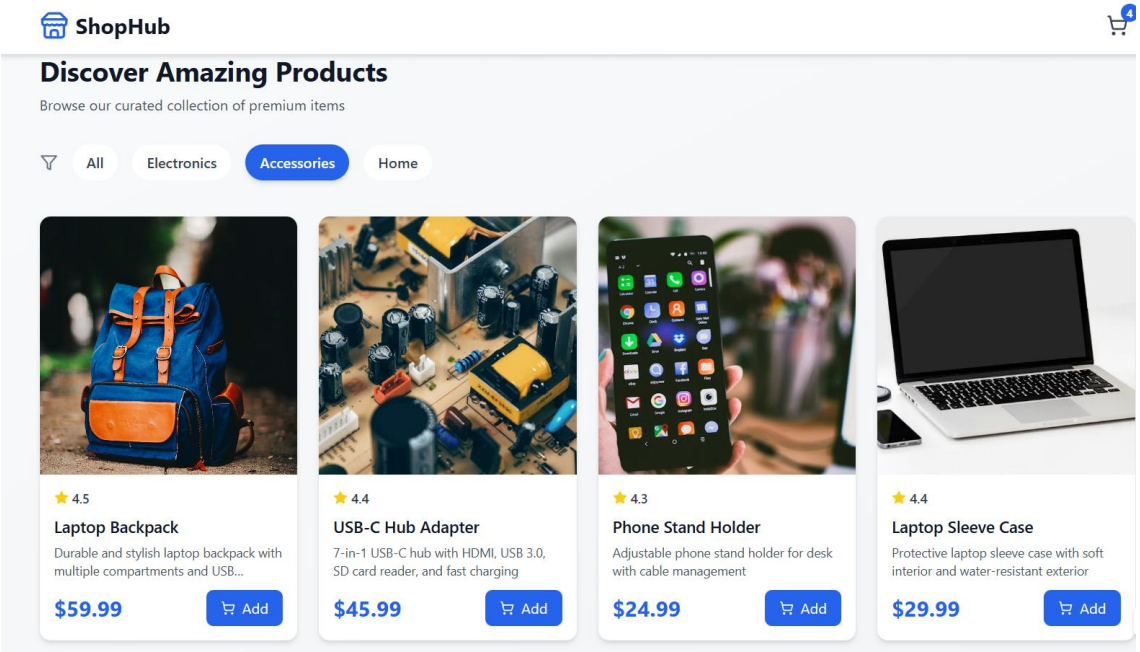
All page:



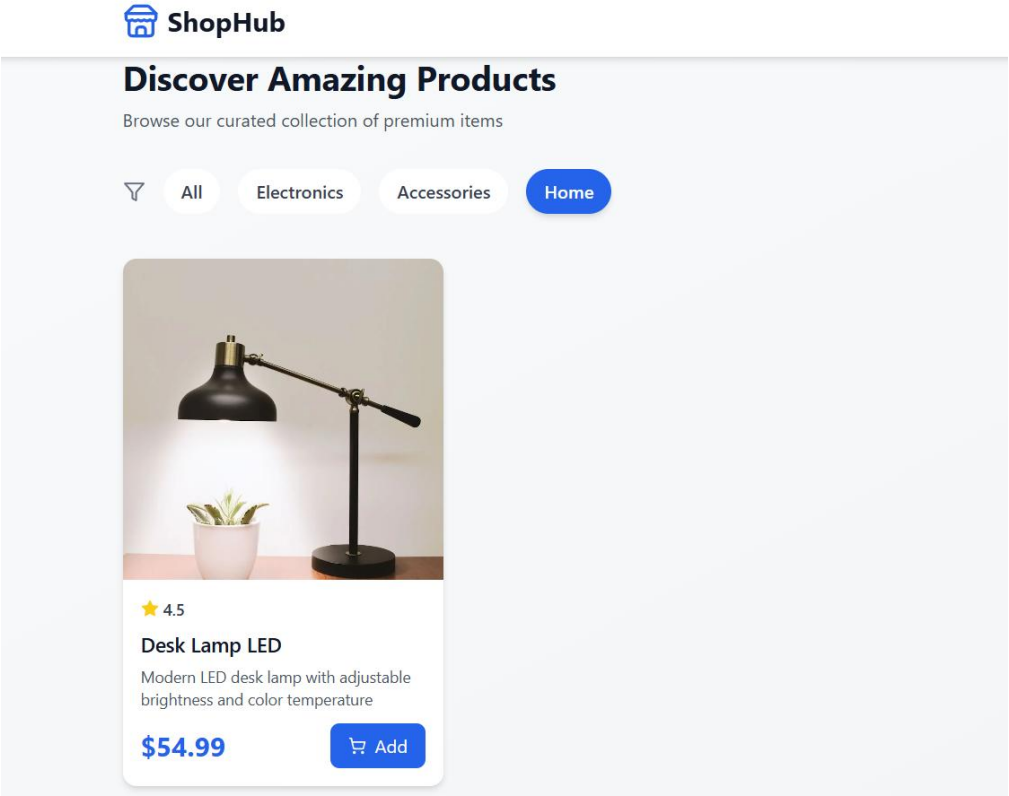
Electronics:



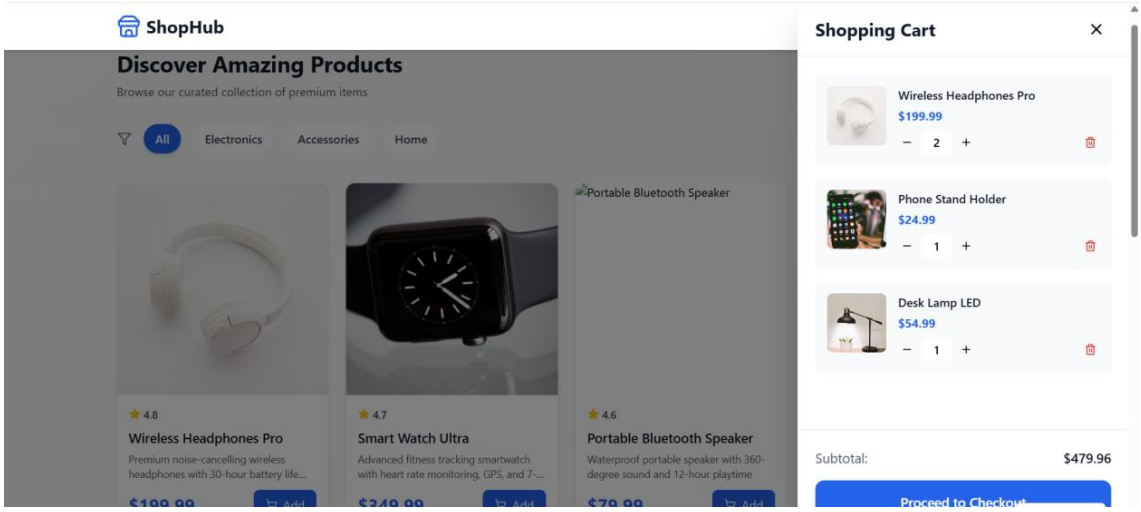
Accessories Page:



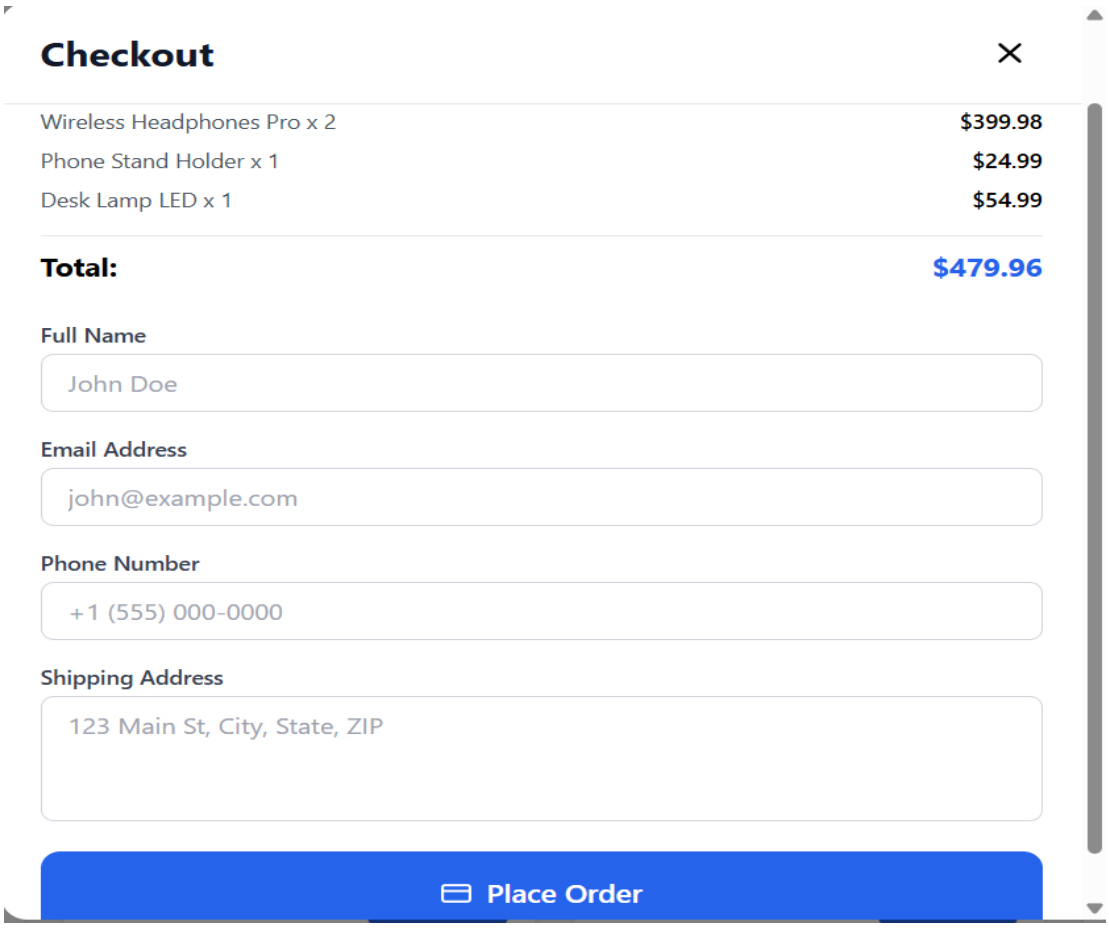
Home:



Cart:



Checkout:



7. Future Advancements:

- Integration of a real backend using REST APIs and a database
- Implementation of user authentication and authorization
- Addition of payment gateway integration for checkout functionality
- Enhanced order history and tracking features
- Improved UI/UX with animations and advanced Material components
- Implementation of state management using NgRx or similar libraries

8. Conclusion

The Product Catalog and Shopping Cart System successfully demonstrates the practical use of Angular and TypeScript in building a modern front-end web application. By implementing core Angular concepts such as components, services, routing, and reactive programming, the project provides hands-on experience in developing scalable and maintainable applications. This project serves as a strong foundation for understanding enterprise-level Angular development and can be extended further to build a complete e-commerce solution.

9. References:

Angular Documentation
TypeScript Documentation
Angular Material Documentation
Node.js
Documentation GitHub Repository –
Shopping_System_ShopHub